NPU-Native Neural Networks on AMD XDNA 2

Hugo Penedones

Draft — April 22, 2026

High-throughput residual inference for HIGGS on AMD XDNA 2

Abstract

We study a deliberately narrow question: which neural-network shape best matches the AMD XDNA 2 NPU for high-throughput, low-latency inference on dense collider-style features? We answer it with a forward-only residual MLP whose input projection, 30 residual blocks, and classifier head all stream across the 32 compute tiles of the Strix Point array. On the public HIGGS benchmark, this fully on-array $H = 32$, $L = 30$ pipeline reaches 76.61% test accuracy and sustains 14.08M samples/s wall-clock at batch size 64 on the fixed 500k-example test split, or 2.44M samples/s at the native streaming batch of 8. On the same machine and the same model, a 12-core CPU baseline peaks at 0.98M samples/s, so the on-array path is roughly 14× faster at the best CPU operating point. The design is small enough to understand end to end yet strong enough to support a credible systems case for dense event filtering on consumer hardware. The contribution is a measured co-designed operating regime, not a production trigger deployment, a state-of-the-art HIGGS accuracy paper, or an energy-efficiency study.

1 Introduction

Modern NPUs become most interesting when they stop being treated as "small GPUs" and start being programmed as spatial dataflow machines. Instead of launching one large shared-memory kernel, the programmer assigns a small local job to one tile, hands the activations to the next tile, and repeats until the full network has been traversed. On AMD XDNA 2, the programmer sees a 2D array of compute tiles, each with local SRAM, a VLIW plus SIMD core, a matrix-multiply unit, and explicit routing. The central question is therefore no longer just "what architecture is accurate?", but also "what architecture maps cleanly to many tiny memories connected by explicit streams?".

The focus here is deliberately narrow: a residual MLP for HIGGS, with one residual matrix per tile and a forward-only spatial-streaming deployment on the NPU. The aim is not to survey every model family or every use of XDNA 2, but to isolate one design that can be understood end to end.

HIGGS is a natural benchmark for this study. It is a dense 28-feature binary classification task derived from collider events, introduced by Baldi, Sadowski, and Whiteson as a large-scale signal-versus-background benchmark with roughly 11 million examples [6, 7]. That makes a residual MLP structurally plausible before any hardware argument is made. It also makes the throughput story easier to defend. For example, the CMS Phase-2 Level-1 trigger must inspect detector data at the 40 MHz bunch-crossing rate, produce decisions with a latency budget of about 12.5 μs, and emit accepted events at roughly 750 kHz [9]. Those rates explain why multi-megasample-per-second measurements are interesting here, but they are not used to imply deployment equivalence: the wall-clock repeated-sample result in this paper is 14.08M samples/s at $B = 64$ on a fully on-array $H = 32$, $L = 30$ path, which sits above the accepted-event rate yet

far below the full bunch-crossing stream. The point is to study the same general systems regime of dense features, strict latency sensitivity, and pressure for very high sustained throughput.

This places the paper next to three adjacent literatures. For HIGGS itself, the reference landscape already includes both the original deep-versus-classic baseline framing and the strong tabular baselines now typically led by boosted tree ensembles [6, 13, 14, 15, 16, 17]; attention-based and decision-tree-hybrid deep tabular methods such as TabNet and NODE have also been evaluated on this benchmark [21, 22]. Comprehensive surveys of machine learning for particle physics provide broader context for where such dense-event classification sits within the HEP community [23]. For low-latency collider inference, substantial prior work has targeted FPGA-based trigger deployments and related co-design flows; the hls4ml open codesign toolchain makes the path from a trained model to firmware particularly direct [10, 19, 9]. At the accelerator level, the closest precedent is the spatial/dataflow tradition, where local SRAM and explicit movement constrain the network shape as much as raw arithmetic throughput does; purpose-built DNN accelerators and the Google TPU family both illustrate how data movement can become the primary design constraint rather than arithmetic throughput [11, 12, 20]. The claim here is narrower: given an existing benchmark, existing consumer NPU hardware, and an existing compiler/runtime stack, what is the simplest residual design that maps cleanly enough to the array to make a credible end-to-end systems case?

2 The hardware model we program against

Figure 1 shows the physical layout targeted here. The Ryzen AI 9 HX 370 exposes 32 compute tiles arranged as 8 columns by 4 compute rows. Below them sit 8 MemTiles, and below those 8 shim tiles that connect the array to host-visible memory and runtime control.

Property	Why it matters here
32 compute tiles (8 columns × 4 rows)	Depth maps naturally to tile count; the retained deployment uses one tile for the embed, 30 for the residual body, and one for the head.
About 64 KB local SRAM per compute tile [4, 3]	Parameters must stay small enough to live locally; that strongly constrains hidden width.
8×8 MMUL-friendly blocked execution	Widths and batch sizes should be divisible by 8, and host-side packing must respect tiled layouts.
Explicit FIFOs, DMA, and routing	Data movement is part of the design, not an afterthought hidden behind a cache hierarchy.
IRON / MLIR-AIE / XRT software stack	The implementation programs the array close to the hardware model rather than through a high-level opaque runtime.

Table 1: Only a few hardware facts matter for this paper. The most important is not the marketing TOPS number, but the combination of tiny tile memories and explicit data movement.

These constraints shape the model directly. The hidden body is kept square so that each tile stores one $H \times H$ residual weight matrix. Widths are kept as multiples of 8 to match the AIE MMUL datapath. The retained deployment places the entire inference path on the array: the first tile runs a padded $32 \to H$ embed, the next 30 tiles run the residual body, and the last tile runs a padded $H \to 8$ head, leaving only small pad/strip operations on the host.

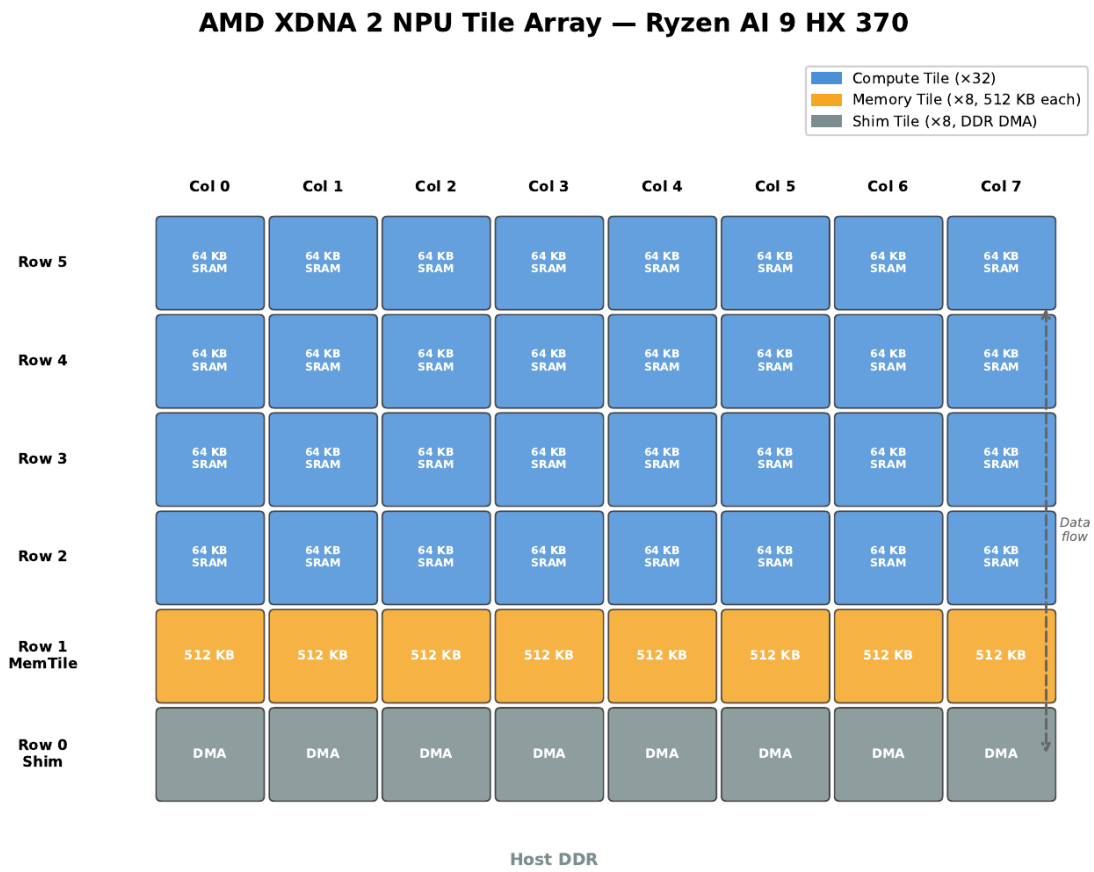

Figure 1: Physical layout of the AMD XDNA 2 NPU used here. The retained design uses all 32 compute tiles: one for the $28 \to H$ embed, 30 for the residual body, and one for the $H \to 2$ classifier head.

3 A residual network that matches the array

The network studied here is intentionally minimal.

Notation

We use H for hidden width, L for the number of residual blocks, and B for batch size. In the deployment path studied here, one residual block maps to one compute tile. We use $\to$ for layer mappings (for example $H \to 2$ classifier) and $\times$ for matrix shapes (for example $H \times H$ weights).

Model definition

$$h_{\ell+1} = \mathrm{ReLU}(h_\ell W_\ell) + h_\ell, \qquad \ell = 0, \ldots, L-1.$$

The skip connection follows the convention introduced by He et al. [18]; each residual matrix W_ℓ is assigned to one compute tile. Activations enter the array, flow through the tiles in a snake, and return to the host only after the full pipeline has completed. In the retained deployment path:

- the first compute tile runs a padded $32 \to H$ input projection that maps native 28-feature HIGGS events into the hidden width,
- the next 30 tiles run the residual body, one $H \times H$ matrix per tile,
- the last tile runs a padded $H \to 8$ classifier head whose first two output columns carry the HIGGS binary logits.

With $H = 32$ this fills the full 32-tile array. The host only pads the 28-wide input to 32, moves the stacked batch across the shim, and later strips the two useful output columns; all matrix

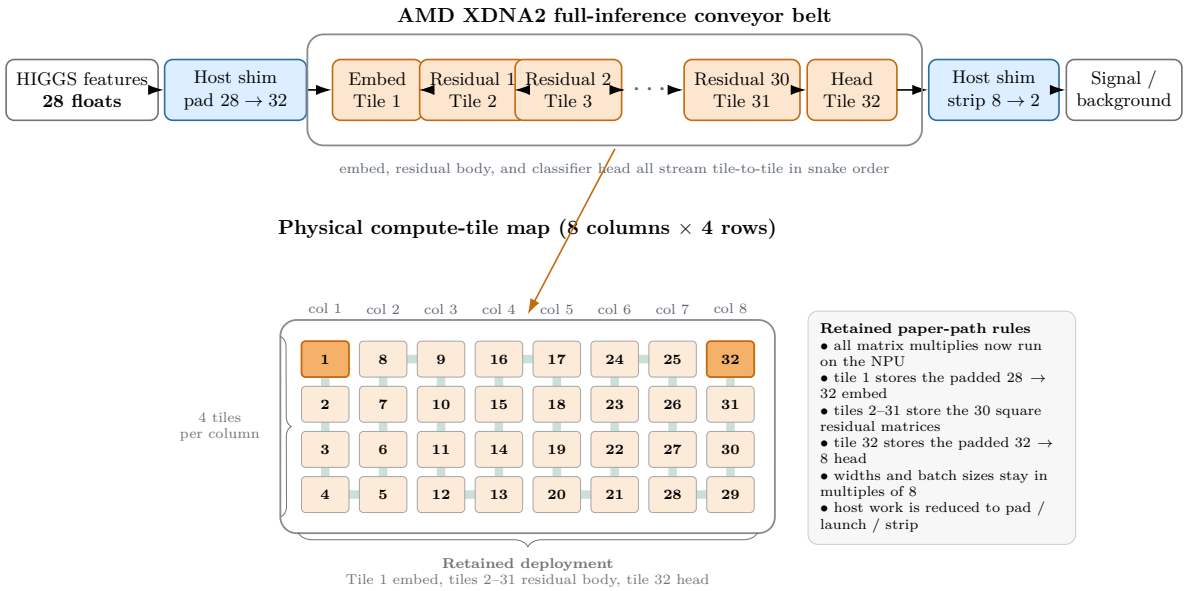

Figure 2: Model-to-hardware mapping for the retained HIGGS/XDNA 2 deployment path. The NPU runs the full inference pipeline: one tile for the $28 \to H$ embed, 30 tiles for the $H \times H$ residual body, and one tile for the $H \to 2$ head. The host only pads the input and strips the output columns.

multiplies run on the NPU. Figure 2 makes that deployment contract concrete.

The important design choice is not just "use an MLP", but "use an MLP whose depth and width map directly to the array's tile structure". That keeps the design simple enough to analyze and direct enough to deploy.

4 From Python model to NPU artifact

The toolchain is deliberately short. `aie_kernels/matmul_relu_skip.cc` defines the tile-local residual kernel, while `aie_kernels/matmul_bias.cc` is compiled into two archives for the embed and head linear stages. `resmlp/streaming_design.py` turns a chosen (H, B, L) shape into a spatial-streaming MLIR-AIE program that wires embed, residual body, and head into a single snake. `resmlp/streaming_op.py` wraps the compiled XCLBIN and instruction binary as a resident XRT operator. Finally, `resmlp/streaming_infer.py` drives trained checkpoints through that operator and measures wall-clock throughput.

The novelty claimed here is not a new compiler or runtime. IRON and MLIR-AIE already provide the compilation substrate [1, 2]; what is new is the model-to-array mapping, the host/NPU partition, and the end-to-end measurements of that co-designed path on real HIGGS checkpoints.

A key detail of the current forward path is that the residual weights are embedded at compile time. Each throughput run therefore builds a checkpoint-specific streaming artifact. That is not the most general deployment contract. It is, however, the cleanest way to study the geometry, routing, and wall-clock behavior of a resident spatial-streaming design.

5 Training protocol on HIGGS

The data pipeline uses the public `jxie/higgs` mirror of the published train/test splits [8]. The local cache materializer produces a fixed 10.5M-example train split and a fixed 500k-example test split. For model selection, training holds out 100k examples from the train split. That leaves 10.4M examples for optimization. All reported runs use the native 28-dimensional feature vector without any padded-width preprocessing. Hyperparameter search and checkpoint selection use

Model	Training recipe
$H = 32$, $L = 30$ full-NPU	9 epochs on the AMD Radeon 890M ROCm path; AdamW, batch 8192, cosine $3 \times 10^{-3} \rightarrow 10^{-4}$, weight decay 10^{-4}, no label smoothing. Early-stopping plateau on held-out validation.

Table 2: Training recipe for the retained $H = 32$, $L = 30$ checkpoint that the on-array deployment uses end to end.

held-out validation accuracy. The 500k-example test split is used only for final evaluation of already-selected models.

HIGGS is also a good benchmark because it sits at the intersection of two real literatures. In high-energy physics it is a canonical dense-event benchmark [6], situated within a broader programme of applying machine learning to particle-physics classification and reconstruction that is surveyed in [23]. In tabular machine learning it is one of the standard large-data problems on which tree ensembles such as XGBoost, LightGBM, and CatBoost remain strong baselines, while modern deep tabular models—including attention-based and decision-tree-hybrid approaches—are now more competitive but not universally dominant [13, 14, 15, 16, 17, 21, 22]. That means the relevant bar is not "beat every HIGGS leaderboard", but "land in a statistically credible part of the baseline landscape while preserving a deployment shape the NPU can actually exploit". Good-enough accuracy on HIGGS is therefore genuinely worth caring about even if the main contribution of this work remains systems-oriented.

6 Results

6.1 Accuracy on the full public HIGGS split

Table 3 summarizes the full-data accuracy of the retained checkpoint. The key point is that the systems design is not accuracy-indifferent: even this deliberately hardware-friendly residual MLP becomes a credible classifier once it is trained on the full public HIGGS data.

The retained checkpoint reaches 0.851 ROC AUC on the 500k-example test split. That is not a claim of state-of-the-art HIGGS accuracy, but it is firmly inside the credible baseline band reported by [6, 16, 17] for comparably sized dense models, and it shows that the hardware-friendly residual shape is not accuracy-indifferent.

6.2 Throughput of the spatial-streaming deployment

Table 4 reports repeated-sample throughput of the retained $H = 32$, $L = 30$ pipeline over 50M processed samples. The wall-throughput column is end to end: input padding, stacked DMA into the shim tiles, the embed + 30 residual + head stages on the array, the return DMA, and the host strip of the two valid output columns. The kernel column is measured separately with an isolated dispatch+wait calibration on the same compiled bitstream, so the ratio quantifies how much of the pure-kernel ceiling survives end to end.

Reaching the multi-megasample regime on this array required more than simply compiling the model: an earlier serial host loop left wall throughput at a small fraction of the kernel ceiling, and moving the embed and head stages on to the array only helped once the host-side overhead had already been amortized. We describe those steps in Appendix B so that the main text stays focused on the end-to-end result.

6.3 CPU throughput baseline

To put the NPU throughput numbers in context, we measure the same models on a laptop CPU using both PyTorch and a decomposed NumPy path that mirrors the NPU's inference structure (separate embed, sequential residual body, head).

Model	Test acc.	ROC AUC	PR AUC	Log loss
$H = 32$, $L = 30$ full-NPU	76.61%	0.8507	0.8635	0.4759

Table 3: Full-data HIGGS results for the retained $H = 32$, $L = 30$ checkpoint on the public `jxie/higgs` mirror, evaluated on the fixed 500k-example test split.

Configuration	Batch size	Wall throughput	Kernel throughput	Wall/kernel
$H = 32$, $L = 30$ full-NPU	$B = 8$	2.44M samples/s	2.48M samples/s	99%
$H = 32$, $L = 30$ full-NPU	$B = 64$	14.08M samples/s	14.16M samples/s	99.6%

Table 4: Repeated-sample throughput of the retained fully on-array $H = 32$, $L = 30$ pipeline. The same compiled bitstream scales from 2.44M to 14.08M samples/s as the streaming batch grows from 8 to 64, and the wall path stays within 1% of the isolated-kernel calibration at both operating points.

Hardware and software. The baseline machine is the same AMD Ryzen AI 9 HX 370 (12 cores / 24 threads, max 5.16 GHz) used for all NPU measurements, running Linux 6.17. PyTorch 2.10.0 uses 12 intra-op threads (the physical core count); NumPy 1.26.4 is backed by OpenBLAS with the same default thread pool. No explicit thread pinning or frequency governor tuning was applied; the measurements reflect the default out-of-the-box configuration.

Methodology. Each benchmark processes 5M samples of the same $H = 32$, $L = 30$ architecture using random `float32` inputs ($D = 28$). Ten warmup calls precede the timed region. The PyTorch path calls `model.forward()` under `torch.no_grad()`; the NumPy path applies the embed, residual chain, and head as explicit `numpy.matmul` calls. We sweep batch sizes from $B = 8$ (matching the NPU's native streaming batch) through $B = 1024$ to show how CPU throughput scales with batch-level parallelism. The model, weights layout, and input distribution match the deployed NPU pipeline exactly.

Two patterns are worth noting. First, the CPU strongly favors large batches: PyTorch throughput improves by more than 30× going from $B = 8$ to $B = 1024$, because larger batches amortize per-call dispatch overhead and expose enough work for BLAS multi-threading. Second, the NumPy decomposed path is faster than PyTorch at small-to-mid batches (lower per-call overhead) but degrades at $B = 1024$ where cache pressure from the $L = 30$ residual chain outweighs the parallelism gain, while PyTorch's fused ATen kernels keep scaling.

Most relevant to the NPU comparison: at the NPU's native streaming batch size of $B = 8$, the CPU reaches only 147k samples/s at best, while the on-array pipeline sustains 2.44M samples/s—a $\sim$17× factor in the NPU's favor. At each side's best operating point, the CPU tops out at 976k samples/s while the NPU scales to 14.08M samples/s at $B = 64$—a $\sim$14× factor, on the identical model. Within the same commodity chip, pushing the entire residual pipeline onto the on-die array therefore yields more than an order-of-magnitude throughput advantage over the 12 host cores.

Note. Both the CPU baseline and the NPU throughput measurements were obtained on the same AMD Ryzen AI 9 HX 370 host, making the comparison controlled: the two results share the same memory subsystem, OS, and software stack, differing only in whether the residual body runs on the host cores or the on-die NPU array.

Configuration	Batch size	PyTorch CPU	NumPy decomposed
$H = 32$, $L = 30$	$B = 8$	28k samples/s	147k samples/s
$H = 32$, $L = 30$	$B = 64$	192k samples/s	613k samples/s
$H = 32$, $L = 30$	$B = 256$	507k samples/s	976k samples/s
$H = 32$, $L = 30$	$B = 1024$	918k samples/s	336k samples/s

Table 5: CPU throughput baseline for the exact retained $H = 32$, $L = 30$ architecture, measured on the AMD Ryzen AI 9 HX 370 host (12 cores) used for all NPU experiments. PyTorch uses 12 intra-op threads; NumPy uses OpenBLAS with default threading. All runs process 5M samples with random `float32` inputs.

7 Discussion

The claim is deliberately narrow but defensible.

- A residual MLP can be shaped to match the AMD XDNA 2 array very directly: one embed + 30 $H \times H$ residual + one head linear, each on its own compute tile, with streamed activations, explicit routing, and checkpoint-specific resident weights.
- That design delivers 14.08M samples/s wall-clock at $B = 64$ on a real collider-style dense benchmark using consumer hardware, while reaching 99.6% of the isolated kernel-throughput ceiling.
- On the same machine and the identical model, the 12-core CPU peaks at 0.98M samples/s, so moving the full pipeline onto the on-die array yields roughly a 14× throughput advantage without any change in compute substrate.
- The same design reaches 76.61% test accuracy and 0.851 ROC AUC on the fixed 500k-example test split, enough to place it inside the credible HIGGS baseline band rather than leaving accuracy as a loose end.

Just as importantly, the paper avoids claims it does not need. It does not present the current model as a production trigger deployment, the current HIGGS result as state of the art, or the present measurements as an energy-efficiency study. The benchmark is public, the hardware is existing AMD XDNA 2, and the compiler/runtime substrate is pre-existing; none of those ingredients are introduced here. The novelty is the co-designed operating regime: a residual MLP whose full forward path — embed, residual body, and classifier head — maps cleanly onto the 32-tile array so that every matrix multiply in inference runs on the NPU.

8 Conclusion and next steps

The main result is simple: a network designed for spatial compute on AMD XDNA 2 can deliver unusually high throughput on real dense-event classification while remaining small enough to understand and validate end to end. The retained model is a fully on-array $H = 32$, $L = 30$ residual MLP that sustains 14.08M samples/s at $B = 64$, stays within 1% of its own kernel ceiling, and reaches a credible 76.61% test accuracy.

The next steps are correspondingly focused. The most direct modeling lever is to push the on-array model harder on accuracy: longer training schedules, stronger regularization, and mild architectural tweaks that preserve the 32-tile footprint. On the systems side, genuine kernel-side gains remain valuable: increasing stream depth toward the per-DMA descriptor limit, overlapping the device-to-host transfer of one chunk with the host-to-device transfer of the next, or revisiting the MMUL tiling so that more compute is packed into each kernel invocation. Energy-efficiency characterization also remains future work. Those are refinements of an already measured pipeline rather than structural changes.

A AMD XDNA 2 background

The main text intentionally keeps only the hardware facts that directly matter for the systems argument. For readers less familiar with AI Engine-class NPUs, Figure 3 gives a programmer's logical view of one compute tile. It is not a physical floorplan. What matters here is that each tile combines local SRAM, a VLIW+SIMD control core, a tile-local MMUL datapath, and explicit DMA / FIFO connectivity. That combination makes weight-stationary streamed mappings natural on this hardware: weights can stay local, activations arrive through explicit buffers, and data movement must be planned rather than delegated to a large shared cache hierarchy [4, 3].

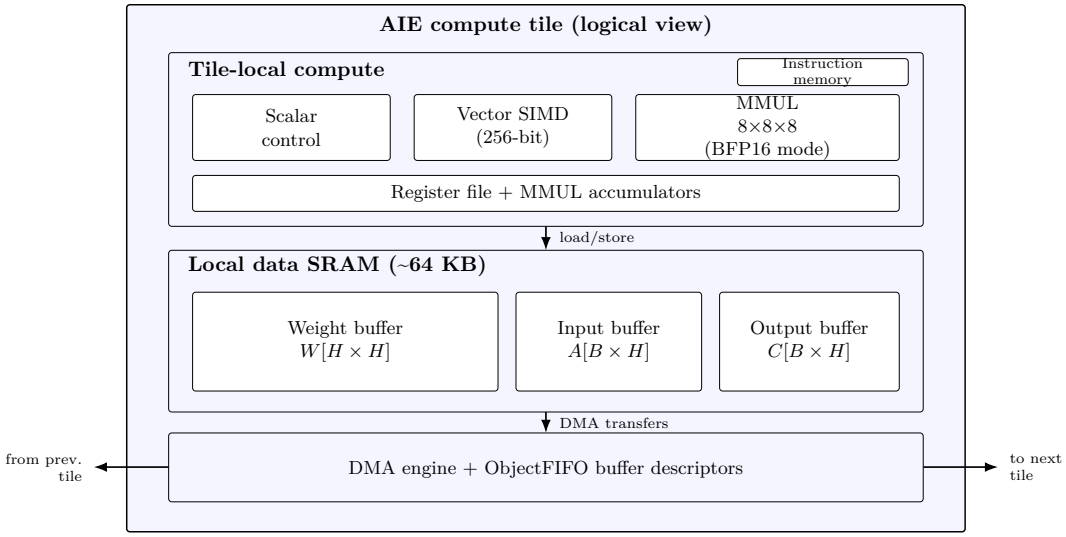

Figure 3: Logical view of one XDNA 2 compute tile from a programmer's point of view. The figure is schematic rather than a physical floorplan. What matters for this paper is the combination of tile-local SRAM, VLIW+SIMD control, MMUL-backed matrix math, and explicit DMA / FIFO movement between tiles.

Table 6 collects the hardware terms that recur most often in the paper. It is meant as a compact glossary, not an exhaustive one, but it gives enough background to follow the deployment discussion without already knowing the AI Engine vocabulary.

Term	Meaning in this paper
AIE tile	One compute tile of the XDNA 2 array: local SRAM, local compute, instruction memory, and DMA connectivity.
MemTile	Memory-only tile between shim tiles and compute rows; useful for routing and on-chip buffering.
Shim tile	Interface tile that connects the array to host-visible memory and runtime infrastructure.
DMA	Hardware transfer mechanism used to move data between host-visible buffers and tile-local SRAM without CPU-managed copy loops.
SIMD	Single Instruction, Multiple Data. One instruction operates on a vector of values at once.
VLIW	Very Long Instruction Word. One instruction can encode multiple parallel operations such as loads, arithmetic, and control.
MMUL	Tile-local matrix-multiply datapath used by the AIE kernel intrinsics; it strongly favors blocked 8×8 layouts.
ObjectFIFO	IRON abstraction for explicit producer/consumer channels between tiles and host boundary stages.
XCLBIN / XRT	The compiled device artifact and the runtime that loads and launches it on the NPU.

Table 6: Compact glossary for the AMD XDNA 2 terms that are used repeatedly in the main text.

B The path to 14M samples/s

The 14.08M samples/s result in Section 5.2 is the output of a short sequence of host-side and graph-side changes, not of a single compilation. We record them here so that a reader interested in reproducing or extending the measurement can see which ingredients mattered.

Baseline gap. An earlier revision of this pipeline used a strictly serial host loop and ran the embed and classifier head on the CPU. Wall-clock throughput was several times lower than the isolated kernel throughput of the same compiled bitstream. Instrumenting the host loop showed that the gap was dominated by Python per-call overhead. For an earlier hybrid $H = 64$, $L = 32$ configuration, one kernel invocation spent roughly 380 µs on the CPU input projection, 64 µs tiling activations, 78 µs detiling the output, and 71 µs on the CPU head, against only 156 µs in the kernel wait itself. The host was the bottleneck, not the array.

Collapse the host loop into stacked numpy. The first change replaced 32 small per-layer Python dispatches per kernel invocation with four stacked numpy operations: one matmul on all $S \times B$ rows for the embed, one reshape/transpose pack, one reshape/transpose detile, and (where still needed) one stacked head matmul. On the same $H = 64$, $L = 32$ checkpoint this collapsed the measured host work from $\sim$590 µs/call to roughly 116 µs/call without touching either the NPU kernel or the network.

Double-buffered host/NPU overlap. The second change exploited the fact that the host work and kernel wait are independent: a compiled bitstream does not care whether the host is already preparing the next chunk. We allocate two device buffer slots per kernel and run a small producer/consumer loop. At steady state the host prepares chunk $N{+}1$ (embed, pack, write_buffer, H2D sync) into one slot while the NPU is still running chunk N on the other slot; the post-wait side of the loop then consumes chunk N's output, again in parallel with the next kernel. Wall time per call becomes close to $\max(\text{host}, \text{kernel})$ rather than their sum.

Calibrate the kernel separately. The overlapped loop makes the per-iteration `dispatch+wait` interval no longer a clean kernel-time measurement: it reports whatever the slower side of the pipeline happens to be. We therefore measure kernel throughput with a dedicated isolated dispatch+wait calibration on the same compiled bitstream. The ratio between async wall-clock throughput and that isolated kernel throughput then quantifies how much of the pure-kernel ceiling the host loop gives back.

Move the entire pipeline on-array. With host overhead already amortized, the remaining wall-time cost was the two CPU matmuls that still bracketed the kernel: the $28 \to H$ embed and the $H \to 2$ head. Moving them on-array required shrinking the residual body from 32 to 30 layers so that one tile could host each of the new linear stages. The embed and head are implemented by a single shared `matmul_bias.cc` kernel compiled twice with different (K, N) defines, producing two per-stage archives, and wired into the same snake as the residual body. Because the array now runs the entire forward pass, host work collapses to zero-padding the 28-wide input to 32 and stripping the two valid columns out of the 8-wide head output.

Scale the streaming batch. With the full pipeline on-array, the remaining lever is arithmetic intensity. Going from $B = 8$ to $B = 64$ packs 8× more work into each kernel invocation at identical host overhead and still fits in the $\sim$64 KB tile SRAM budget at $H = 32$. This is the change that turns the 2.44M samples/s figure into 14.08M samples/s, because at $B = 8$ the retained pipeline is already within 1% of kernel-limited and the only way to go faster is to move the kernel-time ceiling itself.

None of these steps change the learned model: the same 76.61%-accuracy checkpoint is used throughout. They change the artifact that is actually dispatched and the way it is dispatched.

C HIGGS physics background

The HIGGS benchmark comes from a standard collider-analysis setting: a rare target process must be separated from much more common look-alike processes using reconstructed event kinematics [6]. The label is binary. In the UCI release, '1' denotes signal and '0' denotes background [7]. Physically, that does not mean the detector has directly "seen a Higgs." It means the reconstructed pattern of visible objects in the event is more consistent with the target signal process than with ordinary background production.

Each row is therefore already one reconstructed collision event, not raw detector data. The low-level variables describe the visible ingredients of the event: one charged lepton, missing transverse momentum, and up to four jets, plus per-jet b-tag indicators [7]. In collider language, p_T is momentum transverse to the beam, η and ϕ encode direction, and missing transverse momentum records the momentum imbalance associated with invisible particles leaving the detector. The high-level variables are invariant-mass combinations such as m_{jj}, m_{jjj}, m_{lv}, m_{jlv}, m_{bb}, m_{wbb}, and m_{wwbb}; they test whether groups of reconstructed objects line up with a plausible underlying resonance or decay chain [7].

That structure is exactly why HIGGS is useful here. The model does not have to solve raw reconstruction. It receives a compact physics summary of each event and must make a fast signal-versus-background decision on dense numeric inputs. In this repository those examples come from the public `jxie/higgs` mirror of the published train/test split [8].

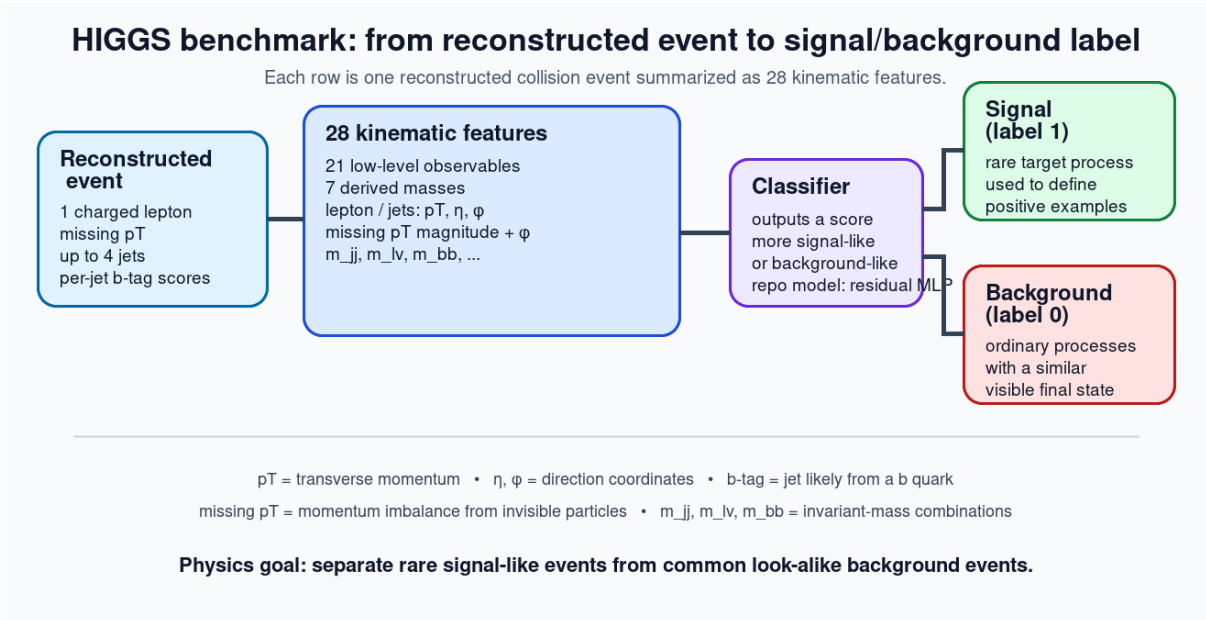

Figure 4: From reconstructed collision event to binary label in HIGGS. The dataset stores lepton, jet, missing-momentum, and derived invariant-mass features; the classifier must separate rare signal-like events from common look-alike background events.

Disclosure of AI-assisted workflow

AI coding and research assistants, including Claude Opus 4.6, GPT-5.4, and Gemini 3.1 Pro, were used extensively during this project to generate code, propose experiments, summarize results, review literature, and critique or draft text. The author curated the research direction, selected the final claims and figures, verified the reported results, and takes responsibility for the content of this paper.

References

[1] AMD, *IRON*.
https://github.com/amd/IRON

[2] Xilinx / AMD, *MLIR-AIE programming guide*.
https://github.com/Xilinx/mlir-aie/tree/main/programming_guide

[3] Linux kernel documentation, *AMD NPU / amdxdna*.
https://docs.kernel.org/next/accel/amdxdna/amdnpu.html

[4] AMD, *AI Engine documentation (programming model and architecture)*.
https://docs.amd.com/r/en-US/ug1273-versal-acap-design/AI-Engine

[5] *AMD XDNA overview*.
https://en.wikipedia.org/wiki/AMD_XDNA

[6] P. Baldi, P. Sadowski, and D. Whiteson, *Searching for Exotic Particles in High-Energy Physics with Deep Learning*.
https://arxiv.org/abs/1402.4735

[7] UCI Machine Learning Repository, *HIGGS dataset*.
https://archive.ics.uci.edu/dataset/280/higgs

[8] J. Xie, *HIGGS* (Hugging Face dataset mirror with published benchmark splits).
https://huggingface.co/datasets/jxie/higgs

[9] R. Ardino et al., *Development and demonstration of the CMS Phase-2 Level-1 trigger Data Scouting system*.
https://doi.org/10.1088/1748-0221/21/01/C01024

[10] J. Duarte et al., *Fast inference of deep neural networks in FPGAs for particle physics*.
https://doi.org/10.1088/1748-0221/13/07/P07027

[11] Y. Chen et al., *DaDianNao: A Machine-Learning Supercomputer*.
https://doi.org/10.1109/MICRO.2014.58

[12] Y.-H. Chen, J. Emer, and V. Sze, *Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks*.
https://doi.org/10.1109/ISCA.2016.40

[13] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*.
https://arxiv.org/abs/1603.02754

[14] G. Ke et al., *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*.
papers.nips.cc/paper/6907-lightgbm

[15] L. Prokhorenkova et al., *CatBoost: Unbiased Boosting with Categorical Features*.
https://arxiv.org/abs/1706.09516

[16] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, *Revisiting Deep Learning Models for Tabular Data*.
https://arxiv.org/abs/2106.11959

[17] L. Grinsztajn, E. Oyallon, and G. Varoquaux, *Why do tree-based models still outperform deep learning on tabular data?*.
https://arxiv.org/abs/2207.08815

[18] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.
https://arxiv.org/abs/1512.03385

[19] J. Duarte et al., *hls4ml: An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices*, Front. Big Data 4, 2021.
https://doi.org/10.3389/fdata.2021.604443

[20] N. P. Jouppi et al., *In-Datacenter Performance Analysis of a Tensor Processing Unit*, ISCA 2017.
https://arxiv.org/abs/1704.04760

[21] S. Ö. Arik and T. Pfister, *TabNet: Attentive Interpretable Tabular Learning*, AAAI 2021.
https://arxiv.org/abs/1908.07442

[22] S. Popov, S. Morozov, and A. Babenko, *Neural Oblivious Decision Ensembles for Deep Learning on Tabular Data*, ICLR 2020.
https://arxiv.org/abs/1909.06312

[23] M. Feickert and B. Nachman, *A Living Review of Machine Learning for Particle Physics*, 2021.
https://arxiv.org/abs/2102.02770